ResumeAI Final Progress Report

***Team Members**: Milan Srinivas, Vrishabh Reddy Leleti, Anurag Bheemappa Gnanamurthy*

What We've Completed So Far

The core pipeline is fully built at this point. We've split the project into three stages: parsing, scoring, and resume generation, with each team member owning one stage.
The parsing layer (Stage 1) is done. We have a resume parser that reads a PDF master resume using PyMuPDF and extracts structured data including contact info (name, email, phone, LinkedIn URL, and city), section headers (experience, education, skills, projects), individual bullet points per experience, company names, job titles, dates, and categorized skills. It handles common formatting quirks like multi-line bullets, two-column layouts, and inconsistent section naming. It also preserves the original categorized skills structure, extracts courses from education entries, and passes through additional resume sections such as Research and Publications, Leadership, and Achievements into the pipeline for full inclusion in the generated PDF. We also have a JD parser that takes raw text pasted from a job posting and pulls out required vs. preferred skills, experience level requirements, and individual requirement sentences. Skills are identified using a custom vocabulary taxonomy of over 200 known technologies with alias-aware matching, so that terms like Apache Kafka and Kafka are treated as equivalent. Both output clean JSON that feeds directly into Stage 2.

For scoring (Stage 2), we have a working TF-IDF baseline that scores each resume bullet against JD requirements using cosine similarity. This gives us a relevance score per bullet based on keyword overlap. We have also integrated sentence-transformers using the all-MiniLM-L6-v2 pretrained model for semantic matching. This is the main AI component that understands meaning beyond exact keyword matches, for example recognizing that "built REST APIs" and "developed backend services" score high similarity despite sharing no words. The semantic scorer includes section-level weighting which adjusts how much weight different resume sections get depending on what the JD emphasizes, with experience weighted at 1.5x by default. The skill gap analysis module identifies required JD skills that are not covered by any resume bullet using alias-aware token matching, and classifies them as covered, partial match, or missing.

For generation (Stage 3), the assembly logic is complete. It takes the full scored output from Stage 2 and the original parsed resume from Stage 1. Experiences are ranked by average bullet relevance and entries below a 0.25 threshold are cut. Bullet counts are allocated by rank where the top experience gets up to 5 bullets and subsequent entries get up to 4, with all original bullets ordered by semantic score rather than hard-filtered. Skills preserve the original categorized structure from the master resume with JD-matched skills promoted to the front of each category. A gap report identifies missing skills and injects them into the generated resume when the flag is used. PDF generation uses Jinja2 templating with a LaTeX template compiled via pdflatex as the primary path, with reportlab as an automatic fallback when pdflatex is unavailable. The reportlab renderer matches the original resume layout exactly, including all

sections: contact with LinkedIn and location, education with courses, categorized skills, experience, projects, Research and Publications, Leadership and Teaching Experience, and Achievements. The multi-JD comparison feature runs one master resume against multiple job descriptions, classifies each bullet as universal, role-specific, or weak, and outputs a seaborn heatmap PNG and a CSV of the full bullet analysis.

What's Left

The full pipeline is complete and verified end-to-end. Evaluation has been run across the four sample job descriptions in the repository using Milan's master resume, confirming that the semantic scorer consistently outperforms the TF-IDF baseline on relevance ranking. The paper outline is ready, and writing begins this week. We have identified a peer-reviewed academic paper from ACL on neural person-job fit matching for the literature review citation.

Plan for the Last Two Weeks

This week we are focused on expanding the evaluation to 15 to 20 real job descriptions from our own internship applications, collecting final comparison numbers for the paper, and writing the Methods and Results sections. Next week is completing the Introduction, Conclusions, and Works Cited, and producing all charts and the multi-JD heatmap figure for the results section. Final days are for code cleanup, inline comments, and submission preparation.

Future Scope

A potential extension is integrating a Retrieval-Augmented Generation (RAG) chatbot that allows users to interact conversationally with their resume and job descriptions. Users could ask questions like "Which of my experiences best matches this JD?" or "How can I improve my resume for backend roles?" The chatbot would retrieve relevant resume bullets and JD requirements from a vector store, then use an LLM to generate personalized advice. Other potential enhancements include activating spaCy NER alongside the existing vocabulary taxonomy for richer skill extraction, fine-tuning the embedding model on resume-JD pairs for improved matching accuracy, adding ATS simulation to predict how Applicant Tracking Systems would score the resume, and supporting multiple output formats such as Word, HTML, and LinkedIn export.

GitHub Repository: https://github.com/WATERMELONBOSS/ResumeAI